

Table of Contents

Assignment 1 1

Assignment 2 2

Assignment 3 3

Assignment 4 4

Assignment 5 5

Assignment 6 7

Generative AI with LLMs

Characters: 17.835

Supervisor: Jesper Wulff

Assignment 1

In the model from chapter 2 in Natural Language Processing with Transformers, their model is fine-tuned for 2 epochs, yielding an F1-score of 0.923 or around 92% for the best model. By training the model for more epochs it is evident that model performance increases. The optimal number of epochs is 3, with an F1-score of 0.937 or around 94%. From the 4th epoch until the 10th and last epoch, the training loss starts increasing, indicating the model is overfitting, hence why F1-scores in this interval have no justification.

Fine-tuning the model for one extra epoch does indeed improve model performance. The F1 score increases significantly in the first few epochs, peaking at epoch 3 (0.937457). After that, the improvements are minimal, with the F1 score remaining fairly stable or slightly declining (e.g., 0.937144 at epoch 8, 0.937815 at epoch 10). This suggests that fine-tuning beyond epoch 3 does not yield substantial gains.

The accuracy follows a similar pattern to the F1 score, peaking at epoch 3 with a value of 0.937 and remaining relatively constant after that. This supports the idea that additional fine-tuning does not significantly improve the model's performance.



	text	label	predicted_label	loss
174	when going to the exam	fear	surprise	2.055779
1596	i leave class feeling more confused every week	fear	surprise	2.033863
146	i shouldnt feel threatened by that	fear	surprise	2.029651
1592	i also get this as another take home message y...	fear	surprise	2.027060
219	im feeling a little vulnerable	fear	surprise	2.026600
1701	i think my mother told me that they feel threa...	fear	surprise	2.026150
209	i feel so terrified to tell her	fear	surprise	2.025640
1699	im feeling rather nervous	fear	surprise	2.024442
1232	i feel a bit frantic today with everything i n...	fear	surprise	2.023443
1676	i always feel scared when i see a cop instead ...	fear	surprise	2.021725

Figure 1: Error analysis

From the error analysis above, sorted on training loss in descending order, there is kind of a pattern in the model mislabeling fear as surprise several times.

By using this information, I tried to trick my model into making an obvious wrong prediction about the following tweet: *“I felt a bit confused, but overall I was really happy”* which in my mind I would label as *“joy”*.

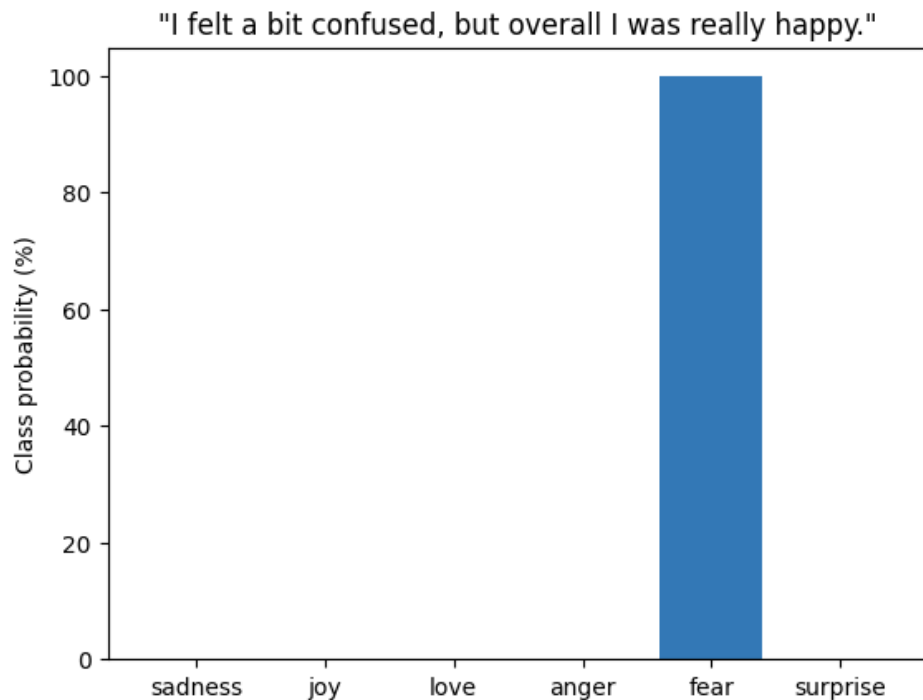


Figure 2: Tweet prediction

The model is convinced that the made-up tweet should be labeled *“fear”*. With this information we can make changes to the dataset to refine it, potentially leading to an increase in performance.

Assignment 2

The text that has been generated through greedy search decoding is coherent but somewhat repetitive and predictable. It’s not overly diverse, as it simply extends the story without introducing much creativity. The text gets truncated mid-sentence, after the output reaches the max length of 128.

The beam search decoding output adds a bit more creative details, like Dr. Jane Goodall and more about the unicorns, making the story feel richer, but it still comes off as a bit predictable due to the method's structured approach. The log probability is -61.50.

The output with temperature = 0.8 sampling introduces more creative and diverse elements, such as a different university and a National Geographic expedition, giving the story a fresh and less predictable feel compared to greedy and beam search. The randomness adds more variety while still maintaining coherence. The log probability is -128.58.

The output with top-k and top-p sampling brings in some imaginative elements like a unique unicorn language and their evolution since the Ice Age. It feels more creative and less predictable. This shows a good balance between randomness and coherence. However, it may introduce slightly more unusual or unexpected details. Log Probability: -124.35

For the given text input about discovering unicorns, sampling with temperature = 0.8 seems to provide the best balance between creativity and coherence. It introduces fresh details, like the involvement of a different university and a National Geographic expedition, while still maintaining a logical narrative that fits with the context of the story. This method avoids the predictability of greedy and beam search while also preventing the excessive randomness seen with top-k and top-p sampling. As a result, it produces a continuation that feels both original and relevant to the input text.

Assignment 3

Having applied few shot prompting to the T5 model using 1 to 4 examples from the dataset, several factors can explain the performance through the ROUGE scores.

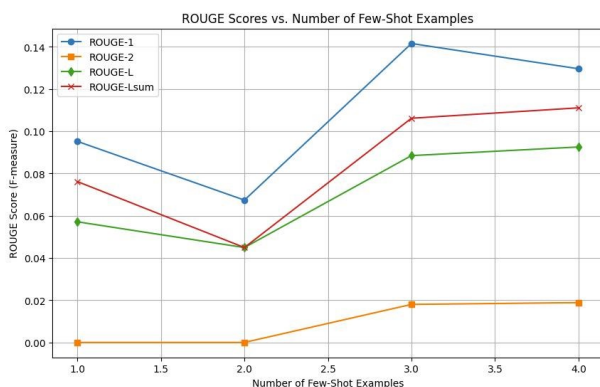


Figure 3: ROUGE-scores

	Number of Examples	ROUGE-1	ROUGE-2	ROUGE-L	ROUGE-Lsum
0	1	0.095238	0.000000	0.057143	0.076190
1	2	0.067416	0.000000	0.044944	0.044944
2	3	0.141593	0.018018	0.088496	0.106195
3	4	0.129630	0.018868	0.092593	0.111111

The ROUGE scores of the T5 model on the first article from the "cnn_dailymail" dataset, already had a quite good indication of strong summarization performance, especially in ROUGE-1 and ROUGE-L. These two measure the overlap of unigrams and the longest common subsequence between the generated summary and the baseline summary (the T5 model in Chapter 6) in

Natural Language Processing with Transformers. ROUGE-2, while lower, still shows the ability of the model to capture bigram overlaps.

The performance of few shot prompting was quite underwhelming from the beginning, as the introduction of one example causes a massive drop in performance compared to the base T5 model. All the ROUGE scores are significantly lower, and the model seems to generally struggle with this additional input context, combined with the lack of any bigram overlap. The model might be confused when prompted with additional information or doesn't know how to utilize the example in a good way, for generating a better summary.

Adding a second example further decreases the performance, suggesting that two examples might be overloading the model, meaning the model does not generalize well.

Introducing a third example actually recovers some performance, indicating a slightly better capture of unigrams, and the capture of bigrams, however not anything meaningful. This recovery of performance could suggest that the model starts to stabilize and is able to process the few-shot examples more effectively

Adding the fourth example we see a slight decrease in ROUGE-1, while the other ROUGE scores increase ever so slightly. Overall the performance is way below the base T5, and the model might have reached a saturation point, considering the plateau in the scores.

To sum it up, few-shot prompting does not improve performance compared to the T5 model on the first article. For future reference, to address problems with performance in few-shot prompting, strategies like curating better examples, reframing prompts, or fine-tuning the model on few-shot tasks could help improve performance.

Assignment 4

To try and improve the performance from Chapter 08 in Natural Language Processing with Transformers, I have experimented with the hyperparameters in the Optuna optimization process. I expanded and refined the following parameters:

For the number of epochs, the book used an interval between 5-10 in running the trials, where I increased the number to 5-15. I did so to broaden the exploration of the optimal number of training cycles.

Unlike the setup in Chapter 8, I introduced `learning_rate` as a tunable hyperparameter with a range between $1e-5$ and $5e-5$. The learning rate provides the model with more flexibility in how

aggressive it is in updating the weights. I included learning rate to increase performance in the model by balancing convergence needs and training stability.

I set alpha to a range between 0 and 1, to control the trade-off between the teacher and the student model's loss. This gives alpha the ability to dictate how much weight is given to each type of loss. By setting it to the said range, a clear and balanced contribution from both hard and soft labels can be maintained. This is crucial to utilize the benefits of knowledge distillation.

For temperature I changed the range from 2-20 (as in the chapter) to 1-15. Changing the temperature allows you to control the smoothness of the logits produced in the teacher model, which has an impact on how easily the student can learn from the teacher model's output. By reducing the limit from 20 to 15, we avoid smoothing the logits too much, possibly obscuring meaningful differences between classes.

Results and Performance Analysis

The adjustments to the hyperparameter settings led to the following changes in performance during benchmarking of the distilled model:

The accuracy on the test set was 0.876, which is an improvement over the book's 0.868. This increase in performance might be attributed to the different and more extended range of epochs, combined with the addition of a learning rate. Increasing the number of training epochs allowed the model to better learn complex relationships within the data, while tuning the learning rate ensured more effective convergence during training.

Overall, the hyperparameter optimization showed an improvement in accuracy, benefiting from increased flexibility in the training phase. This demonstrates that exploration of hyperparameter configurations can be worth doing.

For future reference, one could increase the range of epochs even more, and simultaneously introduce early stopping as to prevent the model from overfitting. This should help enhance model stability and generalization.

Assignment 5

For assignment 5 I tried messing a bit with the hyperparameter optimization to improve performance in the QLoRA fine-tuning.

Chronologically in the code, I started by increasing the **LoRA rank from 8 to 16** and adjusting the **lora_alpha accordingly from 32 to 64**. By increasing both I am effectively improving the model's capacity to learn complex relationships. In doing this, the model can capture more intricate

patterns in the data, thus making it more flexible. The proportional increase in `lora_alpha` makes sure that the scaling of the output stays balanced, so the learning does not overwhelm the training process. Increasing the rank can however lead to a higher computational cost, which diminishes the computational advantage of LoRA.

To help ensure a more stable convergence, I reduced the **learning rate from $2e-4$ to $1e-4$** . In particular when combined with increasing rank and the number of epochs, the convergence of the model should be more stable. Using a smaller learning rate means that the optimizer is adjusting the weights with smaller steps, decreasing the risk of missing any optimal solutions. It also makes learning smoother and more stable. The downsides of reducing or a too low learning rate might result in slower convergence, the need for more epochs because of the smaller steps or vanishing gradients causing the model parameters to stop learning effectively.

I increased the number of **epochs from 10 to 25**, to allow the model to see the data several times, which is necessary when doing fine-tuning for larger models. This is especially true since the learning rate was reduced. By having more epochs it ensures that the model can converge to a lower loss over time. Given that I also utilize gradient accumulation steps, the model effectively performs many weight updates even with a slightly reduced batch size.

Increasing the **weight decay from 0.01 to 0.05** helps prevent overfitting by encouraging the model weights to stay smaller, promoting generalization. This is especially important when training for more epochs and with a larger rank, as it helps to mitigate the risk of overfitting by penalizing overly complex solutions.

My model achieved a validation loss of 1.3292, which is a significant improvement over the QLoRA fine-tuning validation loss of 1.879437 done in the notebook. This shows that the changes I made helped the model to generalize better and effectively minimize the validation loss.

The increased capacity from the rank and `lora_alpha` combined with the lower learning rate, **increased batch size from 4 to 8** and weight decay, enabled the model to explore a larger solution space more effectively while minimizing overfitting risks.

More Training Epochs provided the opportunity for the model to fully exploit the extended capacity and converge to a more optimal state.

The combination of these adjustments creates a balance between flexibility, training stability, and prevention of overfitting, which likely explains the better performance in my model.

Assignment 6

For the question “What is Task Decomposition?” I tried multiple configurations to see how it affected my RAG.

✓	Name	Input	Output	Latency	Tokens	Cost
✓	6th run	What is Task Decomposition?	Task decomposition is...	🕒 1.76s	374	\$0.00007995
✓	5th run	What is Task Decomposition?	Task Decomposition i...	🕒 1.27s	292	\$0.00006495
✓	4th run	What is Task Decomposition?	Task Decomposition i...	🕒 1.62s	515	\$0.00010425
✓	3rd run	What is Task Decomposition?	Task Decomposition i...	🕒 1.66s	304	\$0.00007215
✓	2nd run	What is Task Decomposition?	Task Decomposition i...	🕒 1.74s	519	\$0.00010395
✓	1st run	What is Task Decomposition?	Task decomposition is...	🕒 1.98s	685	\$0.00013785

Figure 4: RAG runs

To identify whether I improve or degrade my rag in terms of output, I will use the output from the notebook RAG01 as kind of an index to compare if the different runs have improved or degraded.

For the first run I used the RecursiveCharacterTextSplitter with a `chunk_size = 1000` and `chunk_overlap = 200`. In langsmith I could tell the retriever had returned the 4 most similar documents, without having specified a search parameter with the exact amount of documents it should retrieve. It was the slowest run in terms of runtime and also the run that spent the most tokens and cost the most. The output is very similar to the one in the notebook, which makes sense as I can see that 2 of the documents in the VectorStoreRetriever are about Tree of Thoughts, explaining it in the same fashion, as well as having the same configuration in the RAG pipeline.

The 5th run had the lowest token usage and the lowest cost. This one boggled me a bit, as it was set to CharacterTextSplitter with `chunk_size = 5000`, `chunk_overlap = 150`, and `separator = “ ”`. I suspected this one to spend the most tokens due to the large `chunk_size` and the fact that CharacterTextSplitter may split the content in arbitrary places, not really respecting natural boundaries, leading to less efficient tokenization and the use of additional tokens when recombining chunks of words or sentences. My assumption is that the summarization and tokenization done by the model was quite efficient, partly because of the retriever parameter being set to “k”: 4, meaning the model needed to process less overall content, and partly because of simple words, redundancy and or filler words within the 5000 character chunk, and the model somehow finding a way to extract and synthesize the relevant information without

unnecessary details. The output from the 5th run seemed a little simplified and less elaborate than the index text. Looking at the VectorStoreRetriever in LangSmith it does make sense, as the 4 documents contained less information, also lacking the document about Tree of Thoughts, hence why the output does not reflect that way of decomposing a task.

For the 4th run I used a CharacterTextSplitter with a chunk_size = 400, chunk_overlap = 10, and the retriever parameter set to k = 7. The use of CharacterTextSplitter with a small chunk size (400) leads to more segments, each representing a small part of the document, which often causes inefficiencies in tokenization. Chunk Overlap of 10 is not sufficient to ensure that continuity is maintained across chunks, which makes it harder for the model to capture the full context efficiently. The retriever parameter set to 7 means that 7 different chunks were used, which increases the token count due to potential redundancy in the information being processed. The relatively high token count can be attributed to the combination of small, fragmented chunks and a high retrieval value.